Note: Statements will be proposed in a random order, without references to the considered ethical aspects.

Instructions text:

You will be presented with a series of software implementation scenarios. Each scenario describes a situation in which more than one implementation choice is possible. You will be asked to indicate how likely you would be to choose the implementation described in each scenario.

Before answering, **think of a software system or application context that you know well,** preferably one that you have worked on directly. You do not need to report or describe the system you selected. **Keep the same system in mind throughout the questionnaire** and evaluate each scenario in relation to that context.

For each scenario, **assume that the functional and non-functional requirements do not prescribe a specific choice, that no legal or organizational rule requires one of the alternatives, and that either implementation would satisfy the intended functionality**. Unless otherwise stated, assume that the difference in implementation effort is not substantial.

Imagine that you are responsible for making the implementation decision. If a scenario does not directly occur in the system you selected, consider how you would make an analogous implementation decision in that context.

There are no correct or incorrect answers. We are interested in the implementation you would personally be more likely to choose under the conditions described.

When faced with the following implementation scenarios, how likely would you be to make the described choice?

Statements:
(* denote reverse-keyed statements, i.e., statements for which the likelihood of the described decision goes against the considered ethical aspect)
*Note: when submitted to software developers, this * and its meaning will be removed.*

Fairness

F1 (Demographic attributes for logic)*. A recommendation component ranks content for users. It can be implemented using the user's past interactions alone or by also considering gender and nationality, which are already available in the user profile. Both implementations can provide the required recommendations. *I would include gender and nationality among the input features used for ranking.*

F2 (Differences in validation procedures). A shopping platform requires sellers to verify their identities before activating their accounts. The verification process can be the same for all sellers or include an additional verification step for sellers who indicate certain nationalities, based on patterns observed in previous registrations. Both implementations

can complete the required identity verification. *I would use the same verification process for all sellers.*

F3 (Stereotypes in code/comments)*. An onboarding component includes an optional guided mode with additional instructions, available to all users. A comment explaining the feature can describe it as providing extra guidance for older users, or simply describe when and how the guided mode is activated. Both comments document the same functionality. *I would describe the feature in the comment as providing extra guidance for older users.*

F4 (Different access rules). An application is gradually releasing a new feature to a limited number of users. Previous releases show that users in one age group are more likely to activate and continue using new features. Access can be granted in sign-up order or prioritized for users in that age group to reduce unused invitations. Both approaches respect the rollout limit. *I would keep the same access procedure for all users without prioritizing any group.*

Inclusiveness

I1 (Accessibility). An interface includes controls that can be operated using a mouse. It can be implemented with mouse interaction only, or it can also support keyboard navigation, which requires additional implementation and testing. Both implementations provide the required functionality. *I would support both mouse and keyboard interaction.*

I2 (Inclusion of different linguistic and cultural conventions)*. A form asks users to enter their name. It can accept Latin characters only; therefore, users whose names use other writing systems can enter a transliterated version, or it can support names written in their original scripts, which requires broader input handling and validation. Both implementations allow users to provide their names. *I would keep support limited to Latin characters only.*

I3 (Gender identities)*. A profile form asks users to provide their gender identity. It can use a predefined set of options (e.g., “male”, “female”, “prefer not to disclose”), which simplifies validation and downstream processing, or include additional options (e.g., “non-binary”, “other” option with non-mandatory free text, etc.), which requires handling more input values. Both implementations allow the system to collect the required profile information. *I would use the predefined set of options.*

I4 (Device accessibility). An interface for desktop computers can be designed around the screen resolutions most commonly used, which keeps layout handling simpler, or it can also accommodate older devices with lower screen resolutions, which requires additional layout adaptations and compatibility testing. Both implementations provide the required functionality. *I would design the interface to also accommodate older devices with lower screen resolutions.*

Privacy

P1 (Data collection). An account registration form requires an email address and password to create an account. It can collect only this information or also ask for additional information such as name, birthdate, gender, and occupation to support more detailed user

representation. Both implementations provide the required account functionality. *I would collect only the information needed to create the account.*

P2 (Data processing and use)*. A feature suggests nearby content based on the user's location. The implementation can use the city selected by the user to provide general suggestions, or, with the user's permission, use their precise location to provide more geographically accurate suggestions. Both approaches fulfill the feature's purpose. *I would use the user's precise location to generate the suggestions.*

P3 (Data retention). A delivery feature allows users to provide instructions for completing an order (e.g., where to leave a package). The implementation can use these instructions for the current delivery only, or, with the user's consent, retain them to automatically reuse them for future orders. Both implementations support the delivery functionality, but differ in whether users need to re-enter their instructions for subsequent orders. *I would use the instructions for the current delivery without retaining them for future orders.*

P4 (Third-party data transmission)*. An application uses an external analytics service to understand how users interact with its features. The implementation can send individual usage events associated with a user identifier, allowing the service to analyze usage patterns across sessions, or send aggregated usage statistics without individual identifiers, allowing analysis of overall feature usage. Both approaches provide information about how the application is used. *I would send individual usage events associated with a user identifier.*

Transparency and explainability

TE1 (Self-explanatory code structure). A function determines whether an order can be refunded based on several conditions, such as payment status, delivery status, and time since purchase. The logic can be implemented as a single compact conditional expression or by defining named variables for each condition and then combining them. Both implementations produce the same result. *I would use named variables for the individual conditions.*

TE2 (Meaningful comments). A component removes milliseconds from a timestamp before sending it to an external service because that service rejects timestamps with millisecond precision. The conversion can be implemented either without additional documentation or with a comment explaining why the precision is reduced. Both implementations behave in the same way. *I would include the comment explaining the reason for reducing the timestamp precision.*

TE3 (Data request explanation)*. A checkout form requires the user's phone number because the delivery service may need it to complete the delivery. The form can mark the field as required without further explanation or display a short explanation of why the phone number is requested, which adds text to the interface that needs to be maintained. Both implementations collect the information required to complete the order. *I would just mark the field as required.*

TE4 (Validation outcomes explanation). A registration form requires passwords to satisfy specific rules, such as minimum length and character requirements. When a password does

not satisfy the rules, the interface can display a generic message stating that the password is invalid or indicate which requirement was not met. *I would indicate which requirement was not met.*

TE5 (Explanation of action consequences). A user can change the username associated with their profile. Changing the username also changes the profile URL, meaning that previously shared links will no longer work. The interface can ask the user to confirm the change without mentioning this consequence or inform them about it before they confirm. Both implementations perform the same username update. *I would inform the user that previously shared profile links will no longer work.*

Traceability

TR1 (State history tracing). An order-processing component updates an order as it moves through states such as *pending*, *confirmed*, and *shipped*. The system can store only the current state or also store and log each state change together with the time it occurred. *I would log each state change and its timestamp.*

TR2 (Attribution). A shared project allows team members to change its display settings (e.g., layout or color theme). The system can log each change and its timestamp, or also record which team member made the change, which requires storing an additional identifier in the log. Both implementations allow the changes to be tracked. *I would also record which team member made the change.*

TR3 (Failure tracing)*. A background job processes a sequence of operations and may fail before completing them. The system can log only that the job failed or also record the operation being executed and the error returned when the failure occurred, which requires handling and storing additional diagnostic information. Both implementations can report that the job was unsuccessful. *I would keep the log simpler and record the failed job only.*

TR4 (Decision traceability)*. A validation component checks a request against several rules before accepting or rejecting it. The system can store only the final outcome, or also log which validation rule determined that outcome, which requires recording additional decision information. Both implementations perform the same validation. *I would store only the final outcome without logging which validation rule determined it*.

Resource Consumption

RC1 (Redundant computation). A dashboard displays the same aggregate metric in several components. Computing the metric requires a non-trivial aggregation of a set of records. Each component can compute the metric independently when it is rendered, which keeps the components self-contained, or the metric can be computed once and reused across components, which requires sharing the result. *I would compute the metric once and reuse the result.*

RC2 (Unnecessary data transfer)*. Several components in an application need the same data during a session. The data does not change during the session. Each component can request the data from the server when it needs it, which keeps the components independent,

or the application can fetch the data once and reuse it locally, which requires managing the shared data. *I would let each component request the data when needed.*

RC3 (Disproportionate dependencies)*. A component requires a set of utility functions. The implementation can use a small library focused on those functions, which requires some additional integration code, or a larger general-purpose library that provides the same functions and is easier to integrate. Both alternatives provide the required functionality, but differ in library size and integration effort. *I would use the larger general-purpose library.*

RC4 (Overhead). A dashboard contains several independent components that display different information. When the data used by one component changes, the interface can update all dashboard components, which keeps the update logic simpler, or update only the components affected by the change, which requires more fine-grained update handling. *I would update only the components affected by the change.*

Possible answers:

- Very unlikely
- Unlikely
- Neither unlikely nor likely
- Likely
- Very likely
- Not enough information or experience to judge

Scores

- Very unlikely: 1
- Unlikely: 2
- Neither likely nor unlikely: 3
- Likely: 4
- Very likely: 5

Reverse-keyed items (marked with a *) are valued as 6-<score>